Lesson Activities — 1.4 Data Types, Data Structures & Boolean Algebra

A bank of low-prep, in-lesson classroom activities for OCR A Level Computer Science (H446) Section 1.4 — built for active teaching with mini-whiteboards, chairs and students as the kit.

1.4.1 Data Types

The Human Binary Counter (unplugged human demonstration · 15 min · whole class)

Resources: 8 place-value cards: 128, 64, 32, 16, 8, 4, 2, 1 (write live on A4) **How it runs:** 1. Line up 8 students facing the class, each holding a place-value card — **128 on the class's left, 1 on the right**. Standing = 1, sitting = 0. All start seated (00000000 = 0). 2. **Counting mode:** teach two local rules — the **1s student flips (stand↔sit) on every count**; every other student flips **only when the student on their right goes from standing to sitting** (a carry). Count aloud from 0 to ~10 and watch the ripple: 1, 10, 11, 100, 101… 3. **Conversion mode:** call a denary number; students self-organise (greedy method: does 128 fit? then 64?…), class verifies by summing the standing cards. Do 45, then 200. 4. **Overflow finale:** set the line to 11111111 (all standing = 255) and add 1. The carry ripples left through every student and falls off the end — the "129th" student doesn't exist. What does the register now show? 5. Debrief: 8 bits ⇒ 2^8 = 256 patterns, range 0–255 unsigned; a carry out of the MSB is **overflow**. **Answers / expected outcomes:** Counting mode reproduces 0,1,10,11,100,101,110,111,1000,… correctly if the two flip rules are followed. 45 = **00101101** (32+8+4+1); 200 = **11001000** (128+64+8). Overflow finale: 255 + 1 → the true answer 256 needs 9 bits; the 8-bit line wraps to **00000000** with the carry lost — overflow. **Stretch:** Convert the standing pattern to hex live by splitting the line into two nibbles (e.g. 11001000 → C8) — one hex digit per nibble.

Two's-Complement Rapid Drill (whiteboard drill · 12 min · individual, mini-whiteboards)

Resources: Mini-whiteboards; place-value line −128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 on the board **How it runs:** 1. Write the signed place-value line on the board and the negation mantra: **"invert ALL the bits, then add 1."** 2. Fire the sequence below at ~45 seconds each; students write 8-bit answers and hold up on "3-2-1-show". After each reveal, one student narrates their method in one sentence. 3. Drill sequence: (1) Show +19 in 8-bit two's complement. (2) Now show −19. (3) What denary is 11111111? (4) What denary is 10000000? (5) What denary is 11110110? (6) Show −100. (7) Show −6. (8) What denary is 11100000? (9) 01111111 + 1: what pattern do you get, what denary does it *claim* to be, and what went wrong? (10) State the full range of 8-bit two's complement. 4. Track class accuracy on the board; re-drill any question with <80% success at the end. **Answers / expected outcomes:** (1) **00010011** (16+2+1). (2) invert → 11101100, +1 → **11101101** (check: −128+64+32+8+4+1 = −19). (3) **−1** (−128+127). (4) **−128** (MSB alone). (5) **−10** (−128+64+32+16+4+2 = −128+118). (6) +100 = 01100100 → invert 10011011 → +1 → **10011100** (check: −128+16+8+4 = −100). (7) +6 = 00000110 → **11111010** (check: −128+64+32+16+8+2 = −6). (8) **−32** (−128+64+32). (9) **10000000**, which reads as **−128**: two positives summed to a negative — **overflow** (127 is the max). (10) **−128 to +127**. Common error to catch: forgetting the "+1" after inverting. **Stretch:** Finish with one subtraction — 23 − 19 as 23 + (−19): 00010111 + 11101101 = (1)00000100; discard the final carry → 00000100 = 4.

Shift Shuffle (unplugged human demonstration · 12 min · whole class)

Resources: 8 students holding 0/1 digit cards (write live); a "bit bucket" chair at the right end **How it runs:** 1. 8 students hold a bit pattern; the class reads its denary value first. Start with **00001100**. 2. **Logical left shift by 1:** everyone side-steps one place left; the leftmost bit falls off; a new "0" student joins on the right. Re-read the value. Repeat once more. 3. Reset to **00011001**. **Logical right shift by 2:** two side-steps right; the two rightmost bit-cards land in the bit bucket; 0s join on the left. Re-read and compare with 25 ÷ 4. 4. Reset to **11110000** and announce it is now *signed* (two's complement). First do a **logical** right shift by 1 and read the (wrong) signed value; then rewind and do an **arithmetic** right shift by 1 — the MSB student stays put *and* sends a copy of their value in from the left. Compare results. 5. Debrief: left shift $\times 2^n$, right shift $\div 2^n$; shifted-out bits are **lost**; arithmetic right preserves the **sign bit** for signed division. **Answers / expected outcomes:** 00001100 = 12; one left shift → 00011000 = 24 (×2); again → 00110000 = 48. 00011001 = 25; logical right by 2 → **00000110 = 6** — exactly 25 DIV 4, with the remainder (the shifted-out bits `01` = 1) lost. 11110000 = **−16** signed; *logical* right 1 → 01111000 = **+120** (nonsense — sign destroyed); *arithmetic* right 1 → 11111000 = **−8** = −16 ÷ 2 ✓. Left shifts can overflow if a 1 falls off the top. **Stretch:** Ask the line to compute 12 × 5 using only shifts and one addition ((12 << 2) + 12 = 48 + 12 = 60).

The Floating-Point Washing Line (unplugged human demonstration · 15 min · pairs then whole class)

Resources: Digit cards; a peg/ruler as the moveable binary point; board **How it runs:** 1. Four students hold a mantissa `0 . 1 1 0 0` (the point-holder stands after the first bit); another student holds the exponent card `0010`. 2. Rule on the board: **value = mantissa × 2^exponent** — the exponent tells the point-holder how many places to walk **right** (positive) or left (negative). 3. Class computes: mantissa value first (halves, quarters…), then the walk. Do these in turn: (a) mantissa 0.1100, exponent 0010; (b) mantissa 0.1010, exponent 0100; (c) mantissa 1.0110, exponent 0010 (remind: mantissa is two's complement — negate it to find its size). 4. **Normalisation round:** set the line to mantissa `0.0011`, exponent `0100`. Ask: "Same value, but we're wasting mantissa bits — fix it." Students shuffle the mantissa left two places and must decide what happens to the exponent so the value is unchanged. 5. Debrief: positive normalised mantissas start `0.1`, negative start `1.0`; mantissa length ⇒ **precision**, exponent length ⇒ **range**. **Answers / expected outcomes:** (a) 0.1100 = 0.75; exponent 2 ⇒ 0.75 × 4 = **3** (point walks right twice: 011.00). (b) 0.1010 = 0.625; exponent 4 ⇒ 0.625 × 16 = **10**. (c) mantissa 1.0110 is negative: invert+1 → 0.1010 = 0.625, so mantissa = −0.625; exponent 2 ⇒ **−2.5**. Normalisation round: 0.0011×2^4 = 0.1875 × 16 = 3; shifting the mantissa two left gives 0.1100 and the exponent must *decrease* by 2 to `0010`: 0.75 × 4 = **3** — value unchanged, now normalised (starts 0.1), two extra bits of precision freed at the right. **Stretch:** Give both pairs the same 8 total bits and let one pair choose a 6-bit mantissa/2-bit exponent, the other 4/4 — who can represent 100? Who can represent 2.5625? (Range vs precision trade-off.)

Odd One Out: Bases & Character Sets (odd one out + think-pair-share · 10 min · pairs)

Resources: Board/slide with sets; mini-whiteboards **How it runs:** 1. Show each set; pairs think alone (20s), agree (30s), write the odd one and the reason, hold up. 2. Sets: (a) `0x2A · 42 · 00101010 · 24`; (b) `'A' · 65 · 0x41 · 97`; (c) `integer · real · char · array`; (d) F_{16} `· 15 ·` 1111_2 `· 16`; (e) `01000001 · 'A' · 65 · 0x65`. 3. After each reveal, one pair explains the trap the set was built around. **Answers / expected outcomes:** (a) **24** — the others all equal 42 (0x2A = 32+10; 00101010 = 32+8+2); trap: digit-swapping 42/24. (b) **97** — that's ASCII `'a'`; the others are `'A'` = 65 = 0x41; trap: case changes the code (difference of 32). (c) **array** — a data structure; the rest are primitive data types. (d) **16** — the others equal 15; trap: F is 15, not 16 (hex digits stop at 15). (e) **0x65** — that's denary 101 (= ASCII `'e'`); the others are `'A'` = 65; trap: 0x65 ≠ 65. Draw out: 1 hex digit = 1 nibble = 4 bits; ASCII maps characters to numeric codes ('0' = 48, 'A' = 65, 'a' = 97). **Stretch:** Pairs build one odd-one-out set where the odd item is only findable by converting *all four* items to binary; sets are swapped and solved.

1.4.2 Data Structures

Human Stack vs Human Queue (unplugged human demonstration · 15 min · whole class)

Resources: 6+ students with name cards; one chair marked "top of stack"; a taped "front/rear" corridor for the queue **How it runs:** 1. **Stack round:** students `A, B, C` are *pushed* one at a time onto a single-file stack against the wall (each new arrival stands in front of the last — only the newest is reachable). Class calls the stack pointer position after each push. 2. `pop()` twice — who leaves, in what order? Then `peek()` — a student reads the top name without anyone moving. Then pop until one more pop is attempted on an empty stack: name the error. Push students until the taped area is full and try one more: name that error too. 3. **Queue round:** students `A, B, C` *enqueue* at the **rear** of the corridor. `dequeue()` — who leaves? Enqueue `D`, dequeue again — who leaves, and who is now at the front? 4. Point at the wasted space at the front of the corridor after several dequeues; ask the class how to reuse it without everyone shuffling forward → wrap the rear around (circular queue, `rear = (rear + 1) MOD size`). 5. Debrief: match each round to LIFO/FIFO and real uses (undo/call stack/backtracking vs print spooling/keyboard buffers/scheduling). **Answers / expected outcomes:** Stack: after push A, B, C, the pops leave in order **C, then B** (LIFO); `peek` reads **A** (now top) without removal; popping an empty stack = **underflow**, pushing a full one = **overflow**. Queue: first dequeue releases **A** (FIFO), second releases **B**; front is then **C** with D behind. Students should state ends precisely: push/pop/peek at the **top**; enqueue at the **rear**, dequeue from the **front**; circular queues reuse freed cells via MOD arithmetic. **Stretch:** Give two waiting students "priority 1" badges and re-run the queue as a priority queue — where must they be served relative to the FIFO order?

The Repointing Game: Linked List vs Array (unplugged human demonstration · 15 min · whole class)

Resources: 5 chairs in a row (numbered 0–4); value cards; students' left arms as pointers **How it runs:** 1. **Array round:** five seated students in chairs 0–4 hold values `3, 7, 9, 12, 15`. Demonstrate O(1) access: "chair 3, shout your value!" — instant. Now insert value `8` in sorted position: everyone from 9 onwards must physically shift chairs right (and there had better be a spare chair). 2. **Linked-list round:** the same students scatter randomly around the room. Each holds their value plus points their arm at the *next* node in order; the teacher holds a "HEAD" sign pointing at the first; the last student points at a "NULL" sign on the wall. 3. Traverse: the class chants the values by following arms from HEAD. Ask chair-3's question again: "what's the 4th value?" — the class must walk the pointers to find out (no random access). 4. Insert student `8`: they stand anywhere; exactly **two** pointer changes happen (7's arm now points at 8; 8 points at 9). Delete node 12: one arm repoints past them. Nobody else moves. 5. Debrief table on board: array = static, contiguous, indexed O(1) access, expensive insert; linked list = dynamic (heap), cheap insert/delete by repointing, but sequential traversal only. **Answers / expected outcomes:** Array insert forces shifting every element after the insertion point and needs spare capacity (fixed size, declared up front). Linked-list access to the 4th element requires following 3 pointers from HEAD (no indexing); insertion of 8 changes only node 7's pointer and sets 8's pointer to 9; deletion of 12 just repoints 9's arm to 15 — the "removed" student still physically exists but is unreachable (garbage). Students should name: node = data + pointer, head pointer, null terminator. **Stretch:** Ask how to make traversing *backwards* possible and what it costs (doubly linked list — a second arm each, double the pointer overhead).

Grow-Your-Own BST (unplugged human demonstration · 15 min · whole class)

Resources: 7 number cards: 50, 30, 70, 20, 40, 60, 80; floor space or playground **How it runs:** 1. Students draw the cards and insert themselves *in this order*: 50, 30, 70, 20, 40, 60, 80. Insertion rule chanted by the class each time: "**smaller — go left; larger — go right**", walking from the root until an empty spot is found. Students stand in tree formation, arms as branches. 2. Check comprehension: "Where would 65 go?" (walk it: 50→right, 70→left, 60→right — new right child of 60). 3. **Traversal walks:** a volunteer walks the tree three ways while the class records the sequence: in-order (Left, Root, Right), pre-order (Root, L, R), post-order (L, R, Root). Pause after in-order: what's special about it? 4. Rebuild the tree inserting *sorted* input 20, 30, 40, 50, 60, 70, 80 — what shape appears, and what happens to search speed? 5. Debrief: BST ordering property; in-order gives ascending output; balanced ⇒ O(log n) search, degenerate chain ⇒ O(n). **Answers / expected outcomes:** Tree: root 50; 30 (left: 20, 40); 70 (left: 60, right: 80). 65 becomes the right child of 60. **In-order: 20 30 40 50 60 70 80** (ascending — the BST property guarantees it). **Pre-order: 50 30 20 40 70 60 80** (used to copy a tree). **Post-order: 20 40 30 60 80 70 50** (used to delete a tree / postfix). Sorted insertion gives a right-leaning chain — effectively a linked list, so searching degrades from ~3 comparisons to up to 7 (O(log n) → O(n)). **Stretch:** Search for 45 by walking the original tree and count comparisons (50→30→40→ not found after 3 comparisons) vs a linear search of the 7 values.

Coat-Hook Hash Table (unplugged human demonstration · 12 min · whole class)

Resources: 7 chairs labelled 0–6 (the table); number cards 6, 15, 22, 8, 29 **How it runs:** 1. Announce the hash function: `index = key MOD 7`. The class computes each index aloud before the key-holder may sit. 2. Insert **6** (sits at 6 — no drama). Insert **15**. Insert **22** — the chair is taken! Class must resolve it: agree **linear probing** (walk right one chair at a time to the next empty seat; wrap at the end). Insert **8** — even more probing. 3. **Search demo:** "Find key 8." Class directs: hash it, start at that chair, step right comparing cards until found. Count the comparisons. 4. **Unsuccessful search:** "Is 29 in the table?" Walk the probe chain until an *empty* chair proves absence. 5. Debrief: hashing gives ~O(1) lookup; a **collision** = two keys hashing to the same index; probing (or chaining) resolves it; as the table fills (load factor rises), probe chains grow and speed decays. **Answers / expected outcomes:** 6 MOD 7 = 6; 15 MOD 7 = **1**; 22 MOD 7 = **1** → collision, probes to **2**; 8 MOD 7 = **1** → probes 1, 2 occupied, sits at **3**. Search for 8: hash to 1, compare chairs 1 (15), 2 (22), 3 (8) — found in 3 comparisons. Search for 29: 29 MOD 7 = 1; probe 1, 2, 3 (all wrong), chair 4 is **empty** → 29 definitively absent. Key insight: identical keys always hash identically (deterministic), so search retraces the exact insertion probes. **Stretch:** Re-run resolving collisions by **chaining** instead (colliders queue behind chair 1) and compare the two searches.

Which Structure Am I? Corners (decision corners · 10 min · whole class)

Resources: Corner signs: STACK, QUEUE, TREE/BST, HASH TABLE (centre floor = "something else — say what") **How it runs:** 1. Read a scenario; students walk to the fitting corner; one delegate per corner justifies before the reveal. 2. Scenarios: (a) the undo feature in a text editor; (b) print jobs waiting for a shared printer; (c) storing 100,000 usernames for near-instant login lookup; (d) a company's staff organisation chart; (e) the browser Back button; (f) keypresses buffered while the CPU is busy; (g) keeping customer records so they can be listed in alphabetical order efficiently; (h) a chessboard's 8×8 grid of pieces; (i) a map of cities and the roads between them. 3. After each reveal, ask the losing corners why their structure is *worse* here (not just why the winner is good). **Answers / expected outcomes:** (a) **Stack** — undo reverses the most recent action first (LIFO). (b) **Queue** — first submitted, first printed (FIFO spooling). (c) **Hash table** — ~O(1) lookup by key. (d) **Tree** — hierarchy with one root, no cycles. (e) **Stack** — last page visited is first returned to. (f) **Queue** — keystrokes must emerge in typed order. (g) **BST** — in-order traversal yields sorted order; O(log n) insert/search when balanced. (h) "Something else":

2D array — fixed size, index by [row][column]. (i) "Something else": **graph** — many-to-many connections, cycles allowed (so not a tree). **Stretch:** For (i), ask whether an adjacency matrix or adjacency list suits a sparse road map and why (list — memory-efficient when few edges).

1.4.3 Boolean Algebra

The Circuit Walk (unplugged role play · 20 min · whole class)

Resources: Gate role cards (AND, OR, NOT, XOR) written live; red/green cards or 0/1 cards as signals **How it runs:** 1. Build the circuit **Q = (A AND B) OR (NOT C)** with students: three "input" students A, B, C at one wall; an AND student (fed by A and B), a NOT student (fed by C), and an OR student (fed by AND and NOT) nearer the other wall. Each gate student states their personal rule aloud ("I output 1 only if BOTH my inputs are 1", etc.). 2. Inputs hold up 0/1 cards; signals propagate by walking a card to each gate; each gate computes and holds up its own output. Run these input rows in order: (1,0,0), (1,1,1), (0,1,1), (0,0,0). The class predicts Q on mini-whiteboards *before* the signal arrives at OR. 3. Swap students between roles each row so everyone plays a gate. 4. **Round 2 — build a half adder:** inputs A and B only; one XOR student and one AND student, both fed by A and B. Labels: XOR's output = **Sum**, AND's output = **Carry**. Walk all four input combinations and record the results as a truth table on the board. Ask: "What operation have we just built out of people?" **Answers / expected outcomes:** Round 1 — Q values: (1,0,0): A·B = 0, ¬C = 1 → Q = **1**; (1,1,1): 1 OR 0 → **1**; (0,1,1): 0 OR 0 → **0**; (0,0,0): 0 OR 1 → **1**. Round 2 — half adder table: (0,0)→Sum 0, Carry 0; (0,1)→1,0; (1,0)→1,0; (1,1)→Sum **0**, Carry **1** — i.e. 1+1 = binary 10. Sum = A⊕B, Carry = A·B; it adds two bits but has **no carry-in**. **Stretch:** Bolt on a third input Cin, a second half adder and an OR student to make a full adder; verify the hardest row (1,1,1) → Sum = 1, Cout = 1.

Always–Sometimes–Never: Boolean Claims (always-sometimes-never · 12 min · pairs, mini-whiteboards)

Resources: Mini-whiteboards; statements on board **How it runs:** 1. Pairs classify each claim as Always true, Sometimes true, or Never true — and must **prove** their answer: an identity/law for Always or Never, a pair of concrete A/B values (one that works, one that fails) for Sometimes. 2. Claims: (a) `A + ¬A = 1` (b) `A·B = A` (c) `¬(A·B) = ¬A·¬B` (d) `A + A·B = A` (e) `A ⊕ A = 1` (f) "a NAND gate outputs 1" (g) `A + 1 = 1` (h) `A·(A+B) = B`. 3. Reveal one at a time; a pair presents their proof; the class hunts for counterexamples. **Answers / expected outcomes:** (a) **Always** — complement law. (b) **Sometimes** — true for A=0 (0=0) or B=1 (A=A); false for A=1, B=0 (0 ≠ 1). (c) **Sometimes** — the classic De Morgan's error; correct law is ¬(A·B) = ¬A **+** ¬B. True at A=B=0 (1=1); false at A=0, B=1 (LHS 1, RHS 0). (d) **Always** — absorption. (e) **Never** — XOR of identical inputs is always 0. (f) **Sometimes** — NAND is 1 for every row except A=B=1. (g) **Always** — null/annihilation law. (h) **Sometimes** — LHS simplifies to A (absorption); true when A=B (e.g. 1,1), false at A=1, B=0. **Stretch:** Pairs repair every Sometimes/Never claim into a genuine identity (e.g. (c) → ¬(A·B) = ¬A + ¬B; (e) → A ⊕ A = 0).

Simplification Surgery (spot-and-fix errors · 12 min · pairs, mini-whiteboards)

Resources: Five "student solutions" on the board; mini-whiteboards **How it runs:** 1. Announce: "Five simplifications from last year's mock — some are right, some are wrong. Grade them." Pairs mark each ✓ or ✗, and rewrite any wrong right-hand side. 2. The five: (1) `¬(A + B) = ¬A + ¬B` (2) `A + ¬A·B = A + B` (3) `A·(A + B) = A·B` (4) `¬(¬A·B) = A + ¬B` (5) `A ⊕ B = A·B + ¬A·¬B`. 3. Whiteboards up; tally the class verdict per line, then verify the contested ones together with a quick truth-table row or a named law. 4. Debrief the two big traps: De Morgan's changes the operator **and** negates each variable; and correct-looking answers deserve a truth-table spot check (e.g. test A=1, B=0). **Answers / expected outcomes:**

(1) ✗ — De Morgan's gives ¬(A+B) = **¬A·¬B** (check A=0,B=1: LHS 0, claimed RHS 1). (2) ✓ — genuine identity (A=0 gives B=B; A=1 gives 1=1). (3) ✗ — absorption: A·(A+B) = **A** (check A=1,B=0: LHS 1, claimed RHS 0). (4) ✓ — De Morgan's: ¬(¬A·B) = ¬¬A + ¬B = A + ¬B. (5) ✗ — that RHS is XNOR; XOR = **A·¬B + ¬A·B** (check A=1,B=1: XOR is 0, claimed RHS gives 1). **Stretch:** Pairs simplify `¬(¬A + ¬B) + A·¬B` fully and prove it equals A (De Morgan's → A·B + A·¬B = A·(B+¬B) = A·1 = A).

K-Map Twister (unplugged human demonstration · 15 min · whole class)

Resources: Masking tape 2×4 grid on the floor; cell labels; string or skipping rope for loops **How it runs:** 1. Tape a Karnaugh map on the floor: rows A = 0, 1; columns BC in **Gray-code order 00, 01, 11, 10** (make the class explain why 11 comes before 10 — adjacent cells must differ by one bit). 2. Read out a truth table for F(A, B, C) that is 1 on minterms 0 (000), 1 (001), 4 (100), 5 (101). One student stands in each cell where F = 1. 3. The class must lasso the standing students with string: loops only in powers of two (1, 2, 4, 8), as large as possible, rectangles only, wrapping allowed. 4. For the winning loop, the loopers interrogate the enclosed cells: "Which variable is the same for everyone inside?" — that surviving literal is the answer. Variables that change inside the loop are eliminated. 5. Repeat with a second function that forces a wrap: F = 1 on minterms 0 (000) and 4 (100) plus 2 (010) and 6 (110) — i.e. columns 00 and 10, both rows. **Answers / expected outcomes:** Function 1: the four standing students fill columns 00 and 01 across both rows — one 4-loop; inside it A varies, C varies, but **B = 0 everywhere**, so F = **¬B**. (Verify: minterms 0,1,4,5 all have B=0.) Function 2: columns 00 and 10 are adjacent *by wrapping* — one wrapped 4-loop where B varies and A varies but **C = 0**, so F = **¬C** (minterms 0, 2, 4, 6 all have C=0). Key rules rehearsed: powers-of-two groups, as large as possible, overlap and wrap allowed, each loop kills every variable that changes within it. **Stretch:** Add minterm 7 (111) to function 1 as a lone student — class must decide the best legal grouping (pair it with 5 to get A·C, giving F = ¬B + A·C).

The Human D-Type Flip-Flop (unplugged human demonstration · 10 min · whole class)

Resources: Two large cards per performer pair: "D = ?" and "Q = ?"; teacher's hands as the clock **How it runs:** 1. One student is the **D input**: they may change their card (0/1) whenever they like, and are encouraged to fidget. A second student is the **flip-flop**: they hold the Q card, and may *only* look at D and copy it at the exact moment the teacher **claps** (the rising clock edge). Between claps they must freeze, whatever D does. 2. Run this script, with the class predicting Q before each event: start Q = 0; D shows 1 → **clap**; D changes to 0 (no clap); teacher waits… → **clap**; D flickers 1, 0, 1 rapidly (no clap); **clap**. 3. Ask the class: "What is this student *doing*, in one word?" (Remembering — it's 1-bit memory.) 4. Add three more flip-flop students side by side sharing the same clap — a 4-bit register storing a nibble in one synchronised edge. 5. Debrief: **edge-triggered** (responds only at the clock edge), **sequential logic** (output depends on stored state + inputs, unlike the combinational gates from the Circuit Walk), used in registers, counters and memory. **Answers / expected outcomes:** Script trace: Q starts 0. Clap 1: D=1 → **Q=1**. D drops to 0 between claps → Q **stays 1** (this is the crucial moment — the class should protest if the flip-flop flinches). Clap 2: D=0 → **Q=0**. D flickers 1,0,1 with no clap → Q **stays 0**. Clap 3: D=1 at the instant of the clap → **Q=1**. Students should conclude: the flip-flop stores one bit, updates only on a clock edge, and holds its value between edges — the clock synchronises when data is captured. **Stretch:** Chain two flip-flops (Q of the first feeds D of the second) and clap twice — students trace how the bit marches one stage per clock pulse (a shift register).